```
        if edgeIntersection(edge1, edge2)
then                                    #4
          k ← k + 1
  return k
```

#1 Method `edgeIntersections` takes a graph, and an embedding to validate, and returns the number of intersections between the graph's edges when drawn with current embedding.
#2 Some basic initialization.
#3 Cycle through every pair of edges.
#4 If this pair of edges intersects when drawn, increase the intersections counter by 1.

Like previous method, we abstracted away the actual algorithm checking the edges: this way, depending on the context, you can use one assuming edges are drawn using straight-line segments, or Bézier curves, etc…

16.1.2 Extending to Curve-Line Edges

In the previous section we have added as a constraint that edges were to be drawn as straight-line segments. As a consequence, we were optimizing the embeddings to reduce the intersections of a straight-line drawing, and the total number of intersections could only get as low as the rectilinear crossing number of a graph: as we have seen in chapter 15, there are many graphs for which the rectilinear crossing number is larger than the crossing number (which, in turn, is the absolute minimum for the number of intersections in any planar embedding of a graph).

The restriction to straight-line drawings was not explicitly required in the code, which was kept as abstract as possible. However, if we would like to use curves for the edges, we need to apply at least one change to listing 16.1: we also have to decide how each edge is modeled. This can be done in several ways, the